

Project Proposal - 2.2

Submitted by: Kanishka Patil, Soham Takale, Anish Apparaju, Ishita Kale

Under the Guidance of: Dr Preeti Kale

Time - 3 to 5 days

Input -

Each supported quantum gate is identified by a unique name (NOT1, NOT2, NOT12, HAD, PHAD, CNOT12, CNOT21, CNOTB12, CNOTB21). The user provides the required circuit as an ordered sequence of these gate names.

Example:

```
[NOT1, CNOT12, HAD, NOT2, CNOTB21, NOT1]
```

where each name corresponds to a predefined gate in the gate library. The gates are executed in the same order as specified in the sequence, and a gate may appear more than once.

Methodology -

1) Understand the Structure of the Existing NMR Pulse Programs

Go through the provided Bruker pulse programs and identify the common setup section, the separate code blocks for each supported gate, and the common readout/footer section. The existing gate blocks will be treated as predefined text blocks that can be reused directly.

2) Develop a Gate Library in Python

Create a Python-based library containing the Bruker pulse-code blocks for each supported quantum gate. Each gate will be assigned a unique numerical identifier for accessing its corresponding code.

Components Observed in the Repo -

Gates / Quantum Operations: NOT1, NOT2, NOT12, HAD, PHAD, CNOT12, CNOT21, CNOTB12, CNOTB21

Other Components: PPS, EQSUP, OBSPOP, INIT, pulse durations (`p1`, `p2`, `p3`, `p4`, `p11`, `p14`, `p31`), delays (`d1`, `d2`), power levels (`p11`, `p12`), phases (`ph0`–`ph3`, `ph31`), RF channels (`f1`, `f2`), gradients (`gp2`, `gp3`), coupling constant (`cnst1`), and acquisition/readout instructions

3) Develop the Circuit Sequence Processing in Python

Implement Python code to accept an ordered sequence of gate names such as `[NOT1, CNOT12, HAD, NOT2, CNOTB21, NOT1]` and look up each name in the gate library to retrieve its pulse-code block, preserving the input order and any repeats.

4) Develop the Pulse Program Generator

Implement the logic in Python to retrieve and combine the required gate pulse-code blocks according to the given gate sequence, include the necessary Bruker pulse-program components, and generate the final Bruker-compatible pulse-program file.

5) Test and Validate the Generated Programs

Test individual gates and different gate sequences to verify that the generated pulse programs are correctly constructed and produce the expected results.

Output – A complete Bruker-compatible NMR pulse program generated according to the sequence of quantum gates provided by the user.

Testing -

Basic Cases – For each gate alone and short sequence such as `[NOT1, CNOT12]`, the generated program is checked to contain the correct pulse-code block for each requested gate, in input order, between the unchanged header, parameter and phase definitions, and footer.

Edge Cases – Reversed order, repeated gates, near-identical blocks (`CNOT12` vs `CNOTB12`), empty input, unknown gate names (error, no file produced), and case or whitespace variants are checked to ensure no block is dropped, duplicated, or reordered, and the common components stay intact.

Basic Tests -

#	Check	Expected
0	Output integrity (runs on every output)	Output equals header + bodies in input order (repeats included) + footer, exactly
1	[XYZ]	error "unknown gate", no file produced
2	[NOT1, XYZ]	error, and no partial file written
3	[]	whatever you decided (error or frame only), but tested on purpose
4	Gate name written as "cnot12" or with a trailing space	normalised
5	All gates individually	Valid code blocks are pulled from the library
6	Mixed Gate Signals (including all gates and different lengths – mostly a randomly generated list)	Valid code blocks are pulled from the library

Edge Cases -

#	Input	Expected output
1	[NOT1, CNOT12] [CNOT12, NOT1]	header + body (NOT1) + body (CNOT12) + footer same snippets in the reverse order, so the file must differ from test 2
2	[CNOT12, NOT1, CNOT12]	body (CNOT12) appears twice, with body (NOT1) between them.
3	[CNOT12] and [CNOTB12]	Each contains only its own body. They differ only in the last pulse's phase (ph0 vs ph2), so this catches a mix-up.